Git — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction to Version Control

02 Installation and Configuration

03 Creating a Repository

04 The Three Areas: Working Tree, Staging, Repository

05 Status, Log and Diff

06 Ignoring Files and Undoing Changes

Intermediate (6 topics)

07 Branching and Merging

08 Resolving Merge Conflicts

09 Remotes: fetch, pull, push

10 Tags and Releases

11 Stash

12 Collaboration Workflows

Advanced (6 topics)

13 Rebase and Interactive Rebase

14 Cherry-pick and Revert

15 Reset Modes and Reflog

16 Bisect and Blame

17 Submodules and Worktrees

18 Git Hooks

Master (4 topics)

19	Git Internals
20	Large Repos, Monorepos and LFS
21	Security: Signed Commits and Secrets
22	Best Practices

Beginner Level

01. Introduction to Version Control

Git tracks changes to files so you can go back in time, work in parallel and collaborate. It is distributed: every clone has the full history.

```
git --version
```

02. Installation and Configuration

Install Git, then set your identity and default branch.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

03. Creating a Repository

`git init` starts a repo; `git clone` copies an existing one.

```
git init my-project
git clone https://github.com/user/repo.git
```

04. The Three Areas: Working Tree, Staging, Repository

Edit files (working tree), choose changes with `git add` (staging), save a snapshot with `git commit`.

```
git add .
git commit -m "Add login page"
```

05. Status, Log and Diff

Inspect your work and history.

```
git status
git log --oneline --graph --all
git diff
```

06. Ignoring Files and Undoing Changes

`.gitignore` hides files; `restore` and `reset` undo work safely.

```
git restore file.txt
git restore --staged file.txt
git commit --amend
```

Intermediate Level

07. Branching and Merging

Branches let you work in isolation; merge to combine. Fast-forward vs merge commit.

```
git switch -c feature/login
git switch main
git merge feature/login
```

08. Resolving Merge Conflicts

Edit conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`), then add and commit.

```
git status   # see conflicted files
git add file && git commit
```

09. Remotes: fetch, pull, push

Sync with a remote repository like GitHub.

```
git remote add origin <url>
git push -u origin main
git pull --rebase
```

10. Tags and Releases

Mark versions with tags.

```
git tag -a v1.0.0 -m "First release"
git push origin v1.0.0
```

11. Stash

Temporarily shelve changes.

```
git stash push -m "wip"
git stash pop
```

12. Collaboration Workflows

Feature branches, pull/merge requests, code review; compare GitHub Flow and Git Flow.

Advanced Level

13. Rebase and Interactive Rebase

Rebase replays commits on a new base for a linear history; interactive rebase squashes/edits commits (only on unpublished work).

```
git rebase main
git rebase -i HEAD~3
```

14. Cherry-pick and Revert

Copy one commit to another branch; create a commit that undoes another.

```
git cherry-pick <sha>
git revert <sha>
```

15. Reset Modes and Reflog

`--soft`, `--mixed`, `--hard`; recover lost commits with `reflog`.

```
git reflog
git reset --hard HEAD@{2}
```

16. Bisect and Blame

Find the commit that introduced a bug; see who changed each line.

```
git bisect start
git blame app.js
```

17. Submodules and Worktrees

Embed other repos; work on several branches at once.

```
git worktree add ../hotfix hotfix
```

18. Git Hooks

Run scripts on commit/push (lint, tests); use Husky or pre-commit.

```
.git/hooks/pre-commit
```

Master Level

19. Git Internals

Objects: blobs, trees, commits, refs; content-addressed storage with SHA hashes.

```
git cat-file -p HEAD
```

20. Large Repos, Monorepos and LFS

Shallow and sparse clones, partial clone, Git LFS for big files.

```
git clone --depth 1 <url>
git lfs track "*.psd"
```

21. Security: Signed Commits and Secrets

Sign commits with GPG/SSH; never commit secrets; clean history with `git filter-repo`.

```
git config --global commit.gpgsign true
```

22. Best Practices

Small focused commits, meaningful messages (Conventional Commits), protected main, frequent pulls, never rewrite shared history.

```
feat(auth): add password reset
```

Interview Question Bank

Q1. Git vs GitHub?
Git is the version control tool; GitHub is a hosting platform for Git repositories.

Q2. `git pull` vs `git fetch`?
`fetch` downloads changes; `pull` = fetch + merge (or rebase).

Q3. Merge vs rebase?
Merge preserves history with a merge commit; rebase rewrites commits for a linear history.

Q4. What is HEAD?
A pointer to the current commit/branch.

Q5. `reset` vs `revert`?
`reset` moves the branch pointer (rewrites history); `revert` adds a new commit that undoes changes.

Q6. What is a detached HEAD?
Checking out a commit instead of a branch; new commits are not on any branch.

Q7. How do you undo the last commit?
`git reset --soft HEAD~1` keeps changes; `git revert HEAD` is safe for shared branches.

Q8. What is `git stash`?
Saves uncommitted changes temporarily so you can switch tasks.

Q9. What is a fast-forward merge?
The branch pointer just moves ahead because there are no diverging commits.

Q10. How do you resolve conflicts?
Edit the conflicted files, stage them, and complete the merge/rebase.

Capstone Project Ideas

1. Maintain a personal portfolio repo with branches
2. Practice conflicts with a friend
3. Create a release with tags
4. Set up a pre-commit hook
5. Contribute to an open-source repo